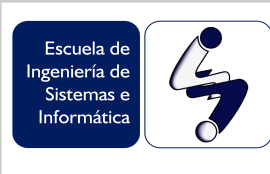


		Universidad Industrial de Santander Facultad de Ingenierías Fisicomecánicas Escuela de Ingeniería de Sistemas e Informática		
Asignatura: Estructuras de Datos y Análisis de Algoritmos		Código: 22955	Grupo: B1/C1	
Docente: Laura Viviana Galvis Carreño		Correo: lavigal@uis.edu.co		

Taller 2. Estructuras de datos lineales y no lineales

Marzo 26 de 2026

Instrucciones: El informe del taller debe subirse al moodle, antes del **14 de Abril a las 8:00 pm**. Informe enviado después de la fecha y hora estipulada no será tenido en cuenta.

El informe debe incluir el código en Java programado en cada uno de los ejercicios cuando se indique y como se indique, así como los resultados de la ejecución en forma de captura de pantalla. Se debe adjuntar un **archivo PDF del informe (resultado de la plantilla de Latex/Overleaf)** con el nombre ***“EDA_B1/C1_Taller2_Nombre_Apellido.pdf”***.

1. ESTRUCTURAS DE DATOS LINEALES

1.1. Listas enlazadas simples: Partiendo del proyecto de listas enlazadas simples programado en clase, desarrollar e implementar en Java un *método que remueva duplicados de una lista ordenada*. Describa con sus palabras, y utilizando diagramas, la lógica del método propuesto. Reportar el código (*lstlisting*) y captura de la ejecución en el informe.

Ejemplo 1 del comportamiento esperado con el método

- Crear una lista enlazada ordenada: INICIO - 3 - 3 - 3 - 6 - FIN
- La salida del método programado debe ser: INICIO - 3 - 6 - FIN

Ejemplo 2 del comportamiento esperado con el método

- Crear una lista enlazada ordenada: INICIO - 1 - 1 - 6 - 6 - 9 - FIN
- La salida del método programado debe ser: INICIO - 1 - 6 - 9 - FIN

1.2. Pilas: Utilizando la estructura de datos tipo pila (*Stack*), desarrollar e implementar en Java un *método que revierta el orden de una secuencia de palabras*. Describa con sus palabras la lógica empleada para desarrollar el método. Reportar el código (*lstlisting*) y captura de la ejecución en el informe. Tener en cuenta que el método:

- Debe funcionar con palabras de cualquier número de caracteres
- Debe funcionar con conjuntos de palabras separadas por un espacio.

Ejemplo del comportamiento esperado con el método

- Crear una pila e incluir la siguiente frase: Estructura de datos
- La salida del método programado debe ser: arutcurtsE ed sotad

1.3. Pilas: Se busca implementar una estructura de datos tipo *pila (Stack)* a partir de estructuras de datos tipo *cola (Queue)*. ¿Con cuántas *colas* podría realizar esta implementación?. Nota: Ningún otro tipo de estructura de datos puede ser utilizado, esto incluye *Arrays*, *LinkedList* o cualquier otra. Describa la lógica de su respuesta detallando el número de *colas* utilizadas así como su función específica dentro de la implementación.

- a. 4
- b. 3
- c. 2
- d. 1

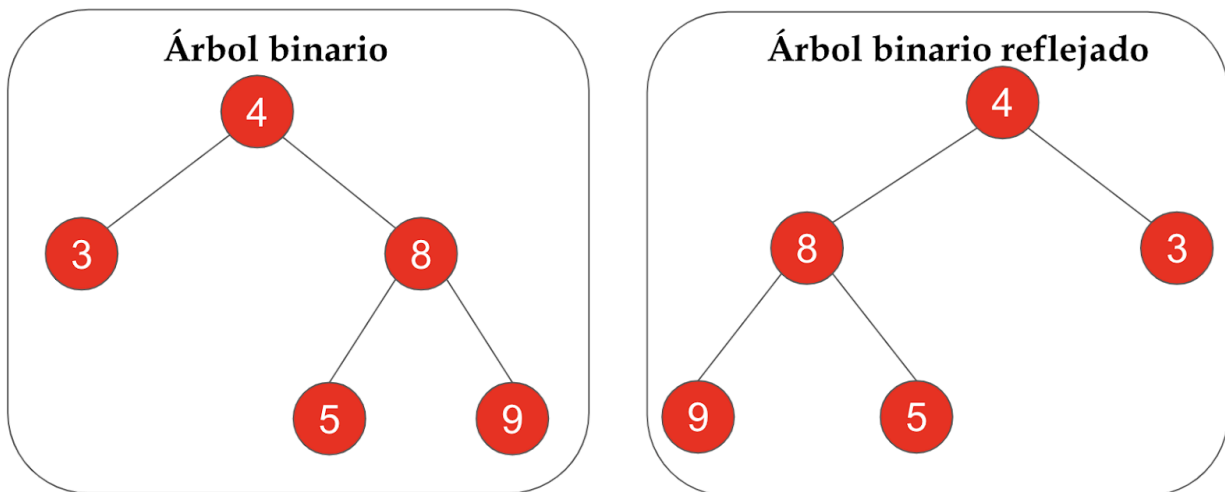
1.4. Colas: Se busca implementar una estructura de datos tipo *cola (Queue)* a partir de estructuras de tipo *pila (Stack)*. ¿Con cuántas *pilas* podría realizar esta implementación?. Nota: Ningún otro tipo de estructura de datos puede ser utilizado, esto incluye *Arrays*, *LinkedList* o cualquier otra. Describa la lógica de su respuesta detallando el número de *pilas* utilizadas así como su función específica dentro de la implementación.

- a. 1
- b. 2
- c. 3
- d. 4

2. ESTRUCTURAS DE DATOS NO LINEALES

2.1. Árboles binarios: Implementar en Java un método que *refleje* un árbol binario. Es decir, aunque se rompe uno de los principios de árboles binarios, en un *árbol binario reflejado* los nodos menores pasan ahora a la derecha de cada nodo padre y consecuentemente los nodos mayores pasan a la izquierda. El método debe aplicarse a un **árbol binario creado inicialmente correctamente**. Reportar el código (*lstlisting*) y captura de la ejecución.

Ejemplo de árbol binario y su correspondiente árbol binario reflejado:



2.2 Árboles binarios: Para el siguiente árbol binario, escribir manualmente las secuencias de salida al recorrerlo utilizando los 4 métodos vistos en clase: recorrido en anchura, recorrido en profundidad *Pre-Orden*, recorrido en profundidad *In-Orden* y recorrido en profundidad *Post-Orden*. Seguidamente, elimine uno a uno los nodos **3, 8, 16 y 12** en ese orden particular, dibuje el árbol resultante y escriba nuevamente las 4 secuencias de recorrido pero del árbol modificado en cada eliminación.

